

Lecture 3 — Scan Conversion I: Points and Lines

Learning outcomes

1. Explain the meaning of scan conversion/rasterization.
 2. Distinguish between continuous geometric descriptions and discrete pixel images.
 3. Convert simple points and lines into pixel locations.
 4. Explain the Digital Differential Analyzer, or DDA, line drawing algorithm.
 5. Explain Bresenham's line drawing algorithm in an intuitive and mathematical way.
 6. Use a decision variable to choose the next pixel in line scan conversion.
 7. Explain why jagged edges occur and how anti-aliasing helps.
 8. Relate scan conversion to games, CAD, fonts, maps, user interfaces, and modern GPU rendering.
-

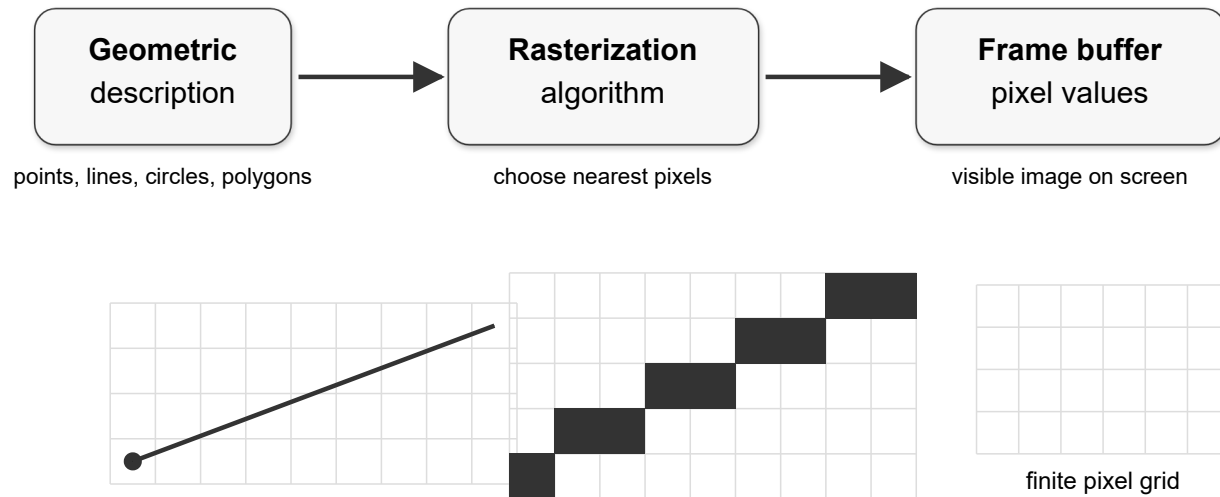
1. The central problem of scan conversion

In mathematics, a line is continuous. It contains infinitely many points. A computer screen, however, is a finite grid of pixels. This creates a fundamental question:

Which pixels should be turned on so that the result looks like the desired geometric object?

This process is called **scan conversion**. It is also commonly called **rasterization**.

Scan conversion: geometry → pixels



A graphics program may define a line using two endpoints:

```
Line from (x1, y1) to (x2, y2)
```

But the display device cannot directly show an abstract mathematical line. The display needs pixel values in the frame buffer:

```
framebuffer[y][x] = color
```

So scan conversion is the bridge between **geometry** and **image representation**.

A very useful mental model

Think of scan conversion as a careful translation problem:

```
mathematical command: draw a line  
hardware reality: color these particular pixels
```

The better the translation, the better the displayed image.

2. Why this lecture is important

Scan conversion may look like an old low-level topic, but it is one of the foundations of computer graphics.

Almost every visible object in a normal raster display eventually becomes pixels. Lines, circles, polygons, text, 3D models, charts, map roads, vector icons, and game characters all need some form of scan conversion.

Applications of scan conversion

In modern GPUs, the word **rasterization** is usually used. A 3D game model is often built from triangles. After transformation and projection, those triangles are rasterized into screen pixels. So the same big idea behind drawing a simple line also appears inside sophisticated real-time rendering pipelines.

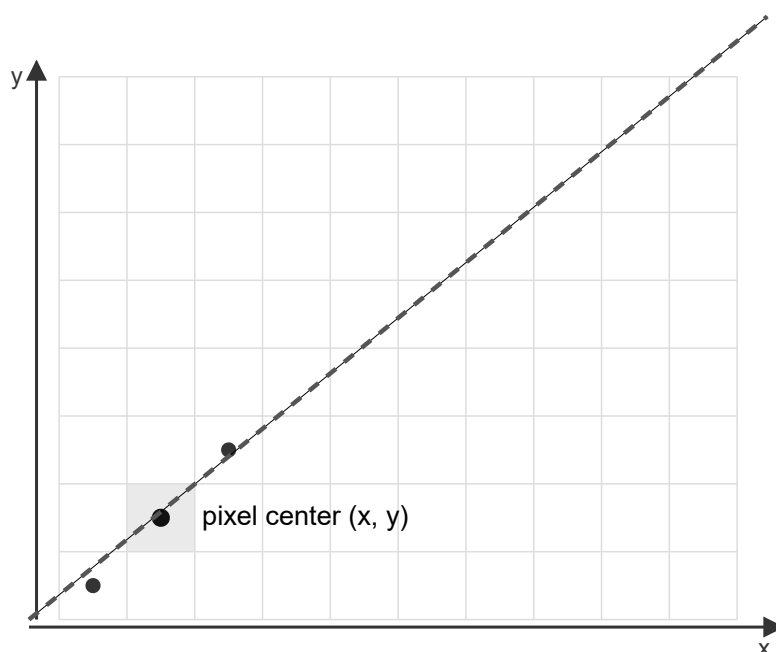
Trivia: why “scan” conversion?

Older raster displays were drawn line by line using a scanning electron beam. The screen was refreshed from top to bottom. The word **scan** therefore comes from the physical process of scanning across display rows. Even though modern flat-panel displays do not work exactly like old CRTs, the idea of a row-by-row raster grid remains central.

3. Pixels, coordinates, and pixel centers

A pixel is not really a mathematical point. It is a tiny square or rectangular area on the display. But in many elementary algorithms we pretend that each pixel is represented by its **center**.

Pixels are areas, but algorithms often use pixel centers



Scan conversion decides which pixel centers best represent a continuous geometric object.

For example, pixel `(5, 3)` may be treated as if it has a center at coordinate `(5, 3)`. The actual display hardware will light up the whole pixel area, but the algorithm decides using a representative coordinate.

This convention is important because line drawing algorithms repeatedly ask:

Which nearby pixel center is closest to the ideal line?

Common coordinate convention

In mathematics, the y-axis usually increases upward. In many image and screen systems, the origin is at the top-left corner and y increases downward.

For algorithm explanation, many textbooks use the mathematical convention. For real graphics programming, always check the coordinate convention of the library or API.

Quick classroom question

If a mathematical line passes exactly between two pixels, which one should be selected?

There is no universal answer. A graphics system must define a consistent rule. Different systems may make slightly different choices at boundary cases.

4. Point scan conversion

A point is the simplest object to scan-convert.

If the point has integer coordinates, such as `(4, 7)`, then we plot the pixel at that coordinate:

```
plot(4, 7)
```

If the point has real-valued coordinates, such as `(4.3, 6.8)`, we usually round to the nearest pixel:

```
plot(round(4.3), round(6.8)) = plot(4, 7)
```

So the simplest point scan conversion rule is:

```
x_pixel = round(x)
y_pixel = round(y)
```

This idea of choosing the nearest pixel will return again and again in line, circle, and curve drawing.

5. The line drawing problem

A line segment is usually given by two endpoints:

```
P1 = (x1, y1)
P2 = (x2, y2)
```

The mathematical equation of a line can be written as:

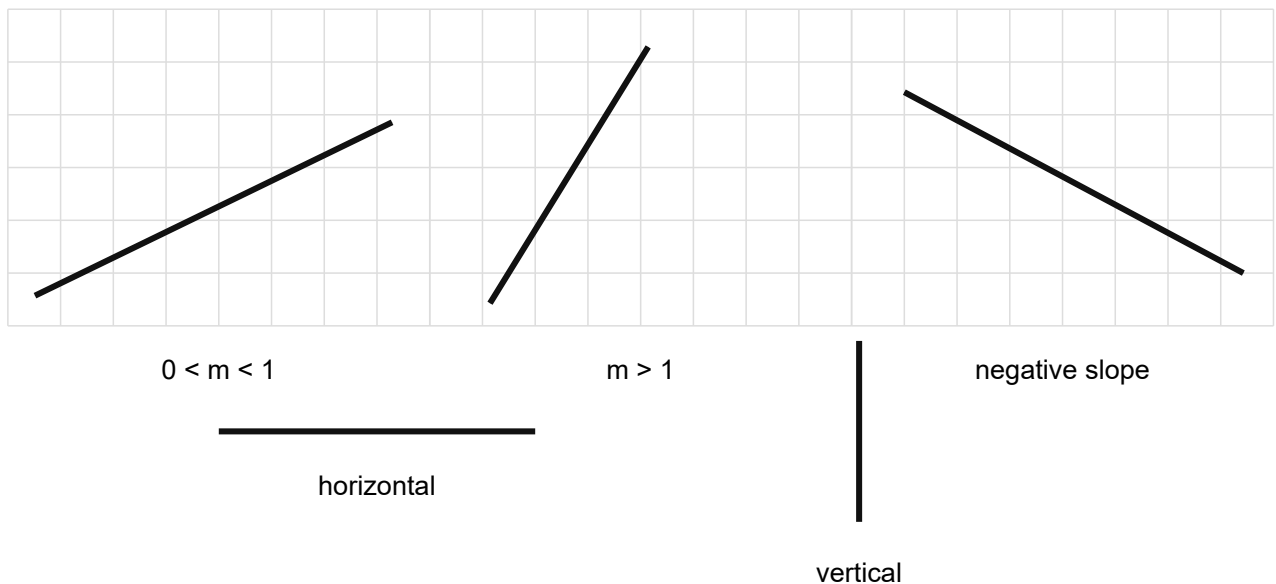
$$y = mx + b$$

where:

$$m = \text{slope} = \Delta y / \Delta x = (y_2 - y_1) / (x_2 - x_1)$$

For a display, we do not need infinitely many points. We need a sequence of pixels that visually approximates the line.

A line algorithm must handle different slopes



A robust line algorithm should handle:

- gentle positive slope,
- steep positive slope,
- negative slope,
- horizontal line,
- vertical line,
- endpoints in any order.

In this lecture, we first focus on the common simple case:

$$0 \leq m \leq 1$$

$$x_1 < x_2$$

That means the line moves more horizontally than vertically. In this case, we can increase x by 1 at every step and decide whether y should stay the same or increase by 1.

Later, the same idea can be generalized to other slopes.

6. Naive line drawing using the line equation

The most direct method is to use:

$$y = mx + b$$

For each integer x , calculate y , round it, and plot the pixel:

```
for x from x1 to x2:
    y = m*x + b
    plot(x, round(y))
```

This is easy to understand, but not ideal for old graphics hardware because it uses floating-point multiplication and rounding in every iteration.

Example

Suppose the line is from $(2, 2)$ to $(10, 6)$.

$$\Delta x = 10 - 2 = 8$$

$$\Delta y = 6 - 2 = 4$$

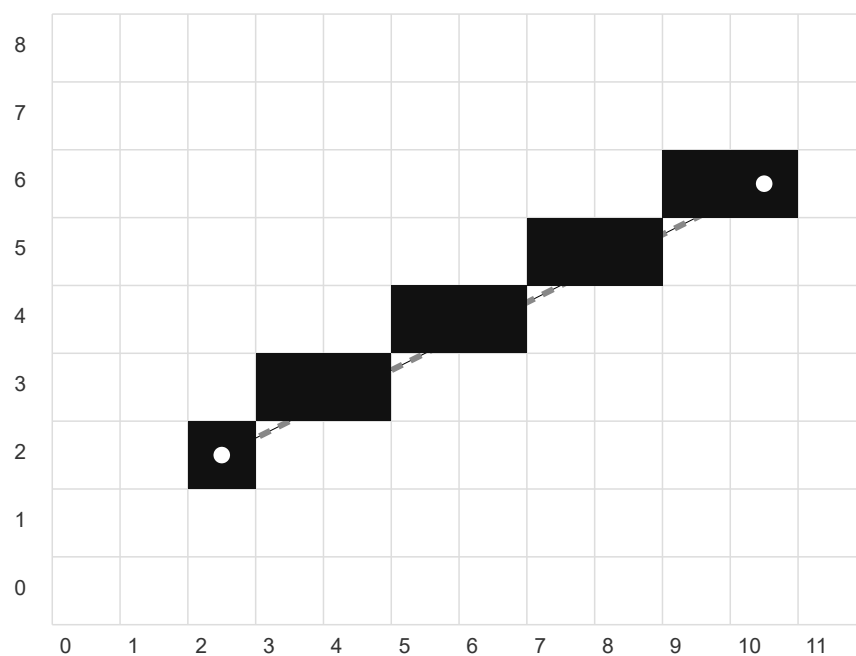
$$m = \Delta y / \Delta x = 4 / 8 = 0.5$$

So every time x increases by 1, the ideal y increases by 0.5.

The ideal mathematical y-values are:

x	ideal y	rounded y	plotted pixel
2	2.0	2	(2, 2)
3	2.5	3	(3, 3)
4	3.0	3	(4, 3)
5	3.5	4	(5, 4)
6	4.0	4	(6, 4)
7	4.5	5	(7, 5)
8	5.0	5	(8, 5)
9	5.5	6	(9, 6)
10	6.0	6	(10, 6)

Worked example: line from (2,2) to (10,6)



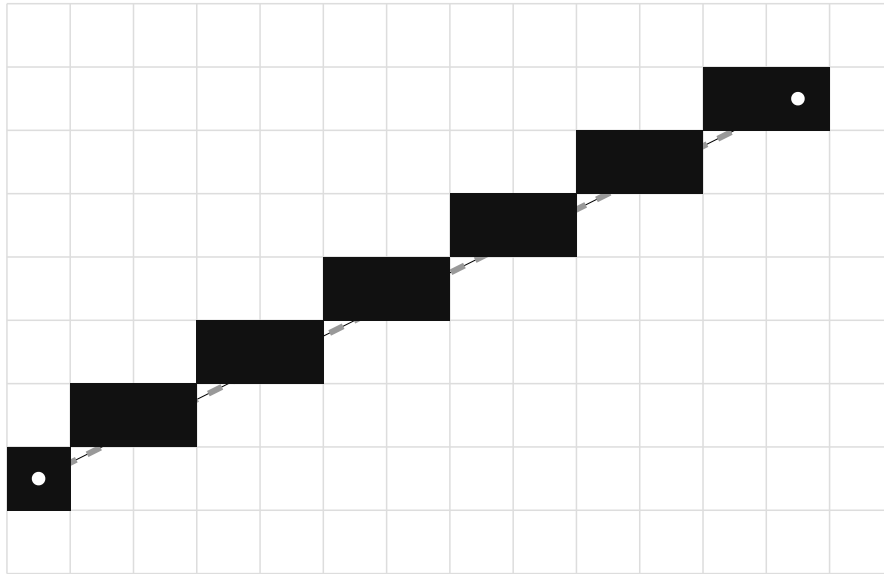
This method works, but computer graphics historically needed faster methods.

7. Digital Differential Analyzer, or DDA

The **Digital Differential Analyzer**, usually written as **DDA**, is an incremental line drawing algorithm.

Instead of computing $y = mx + b$ from the beginning each time, DDA starts from one endpoint and repeatedly adds small increments.

DDA: move in small equal steps and round to pixels



For each step: $x = x + x_inc$, $y = y + y_inc$, then plot $\text{round}(x)$, $\text{round}(y)$.

Main idea

Given two endpoints:

```
(x1, y1), (x2, y2)
```

Calculate:

```
dx = x2 - x1  
dy = y2 - y1
```

The number of steps is usually:

```
steps = max(abs(dx), abs(dy))
```

Then:

```
x_inc = dx / steps  
y_inc = dy / steps
```

Start at (x_1, y_1) . At each step:

```
plot(round(x), round(y))
x = x + x_inc
y = y + y_inc
```

DDA pseudocode

```
DDA_Line(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1

    steps = max(abs(dx), abs(dy))

    x_inc = dx / steps
    y_inc = dy / steps

    x = x1
    y = y1

    for i = 0 to steps:
        plot(round(x), round(y))
        x = x + x_inc
        y = y + y_inc
```

Worked example using DDA

Line from $(2, 2)$ to $(10, 6)$:

```
dx = 8
dy = 4
steps = max(8, 4) = 8
x_inc = 8 / 8 = 1
y_inc = 4 / 8 = 0.5
```

Now we start at $(2, 2)$.

Step	x	y	plotted pixel
0	2.0	2.0	(2, 2)
1	3.0	2.5	(3, 3)
2	4.0	3.0	(4, 3)

Step	x	y	plotted pixel
3	5.0	3.5	(5, 4)
4	6.0	4.0	(6, 4)
5	7.0	4.5	(7, 5)
6	8.0	5.0	(8, 5)
7	9.0	5.5	(9, 6)
8	10.0	6.0	(10, 6)

Strengths of DDA

DDA is:

- simple,
- easy to implement,
- works for all slope cases if `steps = max(abs(dx), abs(dy))`,
- conceptually clear.

Weaknesses of DDA

DDA may require:

- floating-point arithmetic,
- rounding at every step,
- accumulation of small numerical errors.

On modern computers, floating-point arithmetic is usually fast. But historically, this was expensive. That is why Bresenham's algorithm became very important.

8. Bresenham's line algorithm: the intuition

Bresenham's line algorithm is one of the classic algorithms in computer graphics. It was developed at IBM in the 1960s for efficient plotting devices.

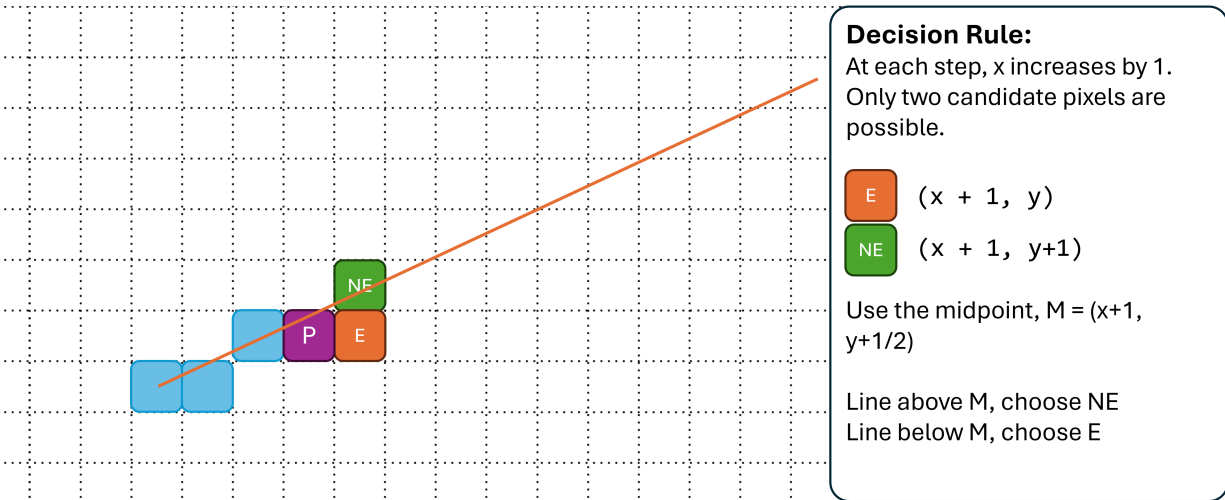
The goal is simple:

Draw a line using only integer addition, subtraction, and comparison.

For the simple case `0 ≤ m ≤ 1`, we move from left to right. At each step, `x` increases by 1. The only question is whether the next pixel should be:

E = East = $(x + 1, y)$
NE = NorthEast = $(x + 1, y + 1)$

Bresenham's key decision: choose East or North-East?



The decision variable tells which candidate is closer to the true line without using floating point arithmetic.

The algorithm chooses the pixel that is closer to the true mathematical line.

The big idea in one sentence

Bresenham avoids calculating the real y value by maintaining a decision variable that tells whether the ideal line has moved far enough upward to increase the pixel y coordinate.

9. Bresenham from the midpoint point of view

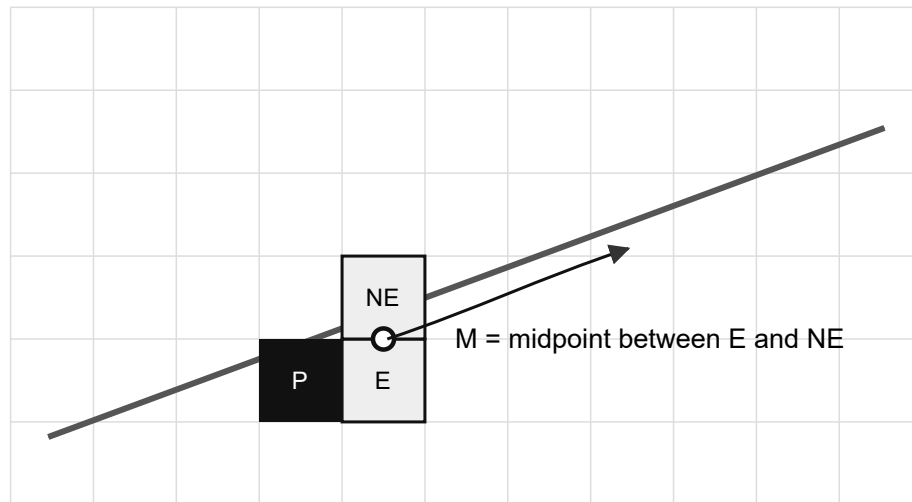
Let the current pixel be (x, y) . The two possible next pixels are:

E = $(x + 1, y)$
NE = $(x + 1, y + 1)$

The midpoint between these two candidate pixels is:

$M = (x + 1, y + 0.5)$

Midpoint test: where is the line relative to the midpoint?



If M is above the true line, choose E. If M is below the true line, choose NE.
The sign of $F(M)$ gives the answer.

If the true line passes **below** this midpoint, then the East pixel is closer.

If the true line passes **above** this midpoint, then the North-East pixel is closer.

So we only need to test the position of the midpoint relative to the true line.

10. Line equation in implicit form

For Bresenham's algorithm, the implicit line equation is very useful.

A line through (x_1, y_1) and (x_2, y_2) can be written as:

$$F(x, y) = dy \cdot x - dx \cdot y + c$$

where:

$$\begin{aligned} dx &= x_2 - x_1 \\ dy &= y_2 - y_1 \end{aligned}$$

The constant c is chosen so that points on the line satisfy:

$$F(x, y) = 0$$

For a point not exactly on the line:

```
F(x, y) < 0    one side of the line
F(x, y) > 0    other side of the line
```

So by evaluating **F** at the midpoint, we can decide which pixel to choose.

But do we need to memorize this derivation?

The most important thing is to understand the decision process:

```
At each x step, choose either E or NE.
The decision variable tells us which one is closer.
```

The derivation explains why the update formulas work.

11. Bresenham update formula for $0 \leq m \leq 1$

For the common case:

```
0 ≤ m ≤ 1
x1 < x2
```

we define:

```
dx = x2 - x1
dy = y2 - y1
```

The initial decision parameter is:

```
p0 = 2dy - dx
```

At each step:

If:

```
p < 0
```

choose East:

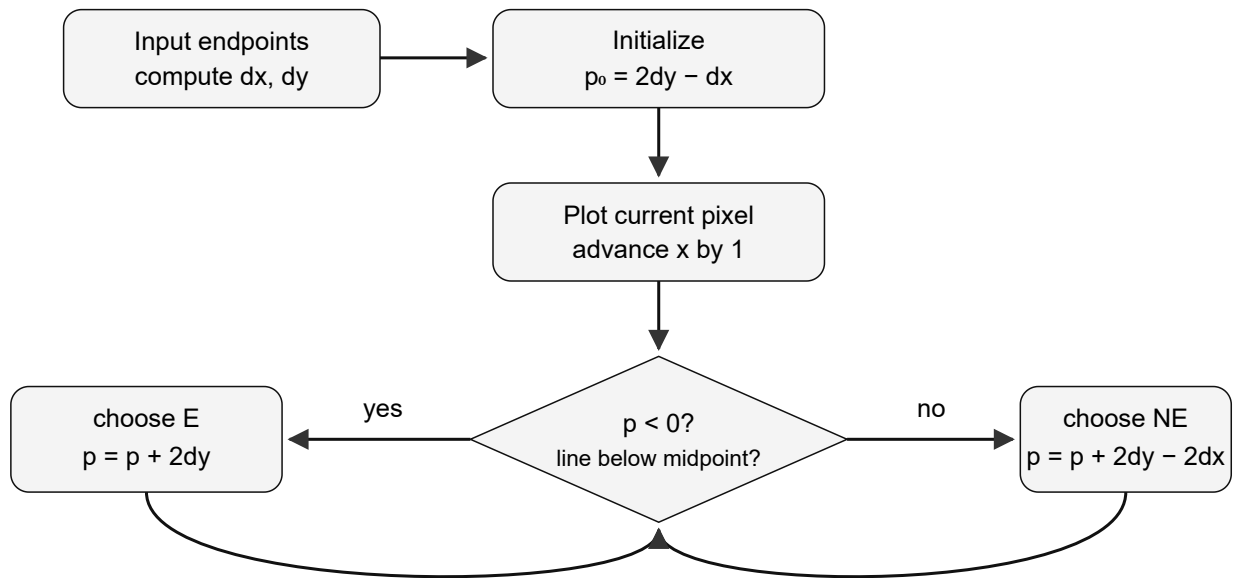
```
x = x + 1
y = y
```

```
p = p + 2dy
```

Otherwise choose North-East:

```
x = x + 1  
y = y + 1  
p = p + 2dy - 2dx
```

Bresenham line algorithm: control flow for $0 \leq m \leq 1$



Bresenham pseudocode for $0 \leq m \leq 1$

```
Bresenham_Line_0_to_1(x1, y1, x2, y2):  
    dx = x2 - x1  
    dy = y2 - y1  
  
    p = 2*dy - dx  
  
    x = x1  
    y = y1  
  
    plot(x, y)  
  
    while x < x2:  
        x = x + 1  
  
        if p < 0:
```

```

        # choose East
        p = p + 2*dy
    else:
        # choose North-East
        y = y + 1
        p = p + 2*dy - 2*dx

    plot(x, y)

```

Important observation

Inside the loop, the algorithm only uses:

```

addition
subtraction
comparison

```

No multiplication inside the loop. No division inside the loop. No floating-point arithmetic inside the loop.

That was a huge advantage on older hardware and remains an elegant example of algorithmic optimization.

12. Worked example: Bresenham line from (2, 2) to (10, 6)

Given:

```

x1 = 2, y1 = 2
x2 = 10, y2 = 6

```

Calculate:

```

dx = 10 - 2 = 8
dy = 6 - 2 = 4
p0 = 2dy - dx = 2(4) - 8 = 0

```

Now apply the rule.

Step	Current pixel plotted	p before decision	Decision	p after update
0	(2, 2)	0	start	0
1	(3, 3)	0	NE	$0 + 8 - 16 = -8$

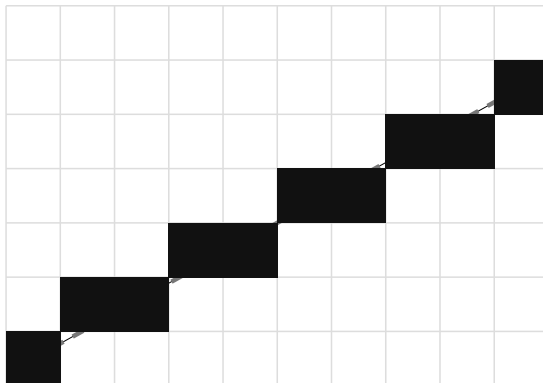
Step	Current pixel plotted	p before decision	Decision	p after update
2	(4, 3)	-8	E	$-8 + 8 = 0$
3	(5, 4)	0	NE	$0 + 8 - 16 = -8$
4	(6, 4)	-8	E	$-8 + 8 = 0$
5	(7, 5)	0	NE	$0 + 8 - 16 = -8$
6	(8, 5)	-8	E	$-8 + 8 = 0$
7	(9, 6)	0	NE	$0 + 8 - 16 = -8$
8	(10, 6)	-8	E	endpoint reached

The plotted pixels are:

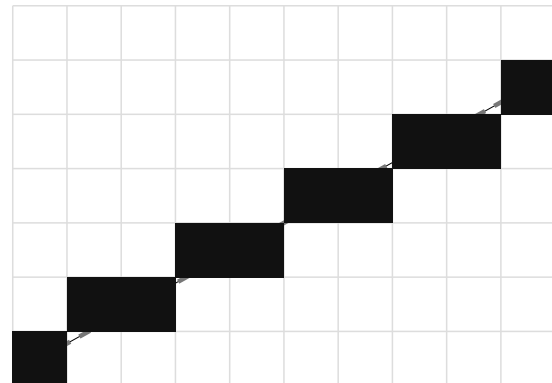
$(2, 2)$, $(3, 3)$, $(4, 3)$, $(5, 4)$, $(6, 4)$, $(7, 5)$, $(8, 5)$, $(9, 6)$, $(10, 6)$

These are the same pixels we obtained earlier using the simple line equation and DDA for this particular example.

Same line, different ways of thinking



DDA: round each step



Bresenham: integer decisions

Often the selected pixels are similar; the difference is computational method and exact decision rules.

13. Why does Bresenham work?

Bresenham works because it converts a geometric closeness problem into an incremental integer decision problem.

For each next column of pixels, there are two candidates. Instead of computing exact distances every time, the algorithm keeps a decision variable `p`.

- If the decision variable says the line is closer to the lower candidate, choose East.
- If it says the line is closer to the upper candidate, choose North-East.

After the choice, `p` is updated by adding a constant. The constants depend only on `dx` and `dy`.

This is the beauty of the algorithm: the difficult geometry is handled once, and then the loop becomes very cheap.

Good way to remember it

For $0 \leq m \leq 1$:

```
Each step must go right.  
Sometimes it also goes up.  
The decision variable tells when to go up.
```

14. Handling other slopes

The version above works directly for:

```
0 ≤ m ≤ 1
```

For other slopes, we adapt the same idea.

If the slope is greater than 1

The line rises faster than it runs. So instead of stepping through `x`, we step through `y`:

```
y = y + 1
```

and decide whether `x` should also increase.

If the slope is negative

The line goes downward as `x` increases. We can still step in the major direction, but the minor coordinate may decrease instead of increase.

If the endpoint order is reversed

We can swap the endpoints so the algorithm moves in a convenient direction.

Practical implementation idea

A complete implementation often handles all cases by:

1. computing `dx` and `dy`,
2. determining the sign of movement in `x` and `y`,
3. selecting the major axis,
4. using an error or decision term to decide when to step in the minor axis.

A compact general version is often called the **integer line drawing algorithm** or a generalized Bresenham algorithm.

15. DDA vs Bresenham

Feature	DDA	Bresenham
Main idea	Increment real-valued x and y	Use integer decision variable
Arithmetic	May use floating point	Integer addition/subtraction/comparison
Ease of understanding	Very easy	Slightly more difficult
Efficiency on old hardware	Lower	Higher
Historical importance	Important	Very important
Conceptual value	Shows incremental drawing	Shows clever discrete decision-making

Which one should we learn first?

DDA is usually easier for building intuition. Bresenham is better for understanding how graphics algorithms are optimized for discrete hardware.

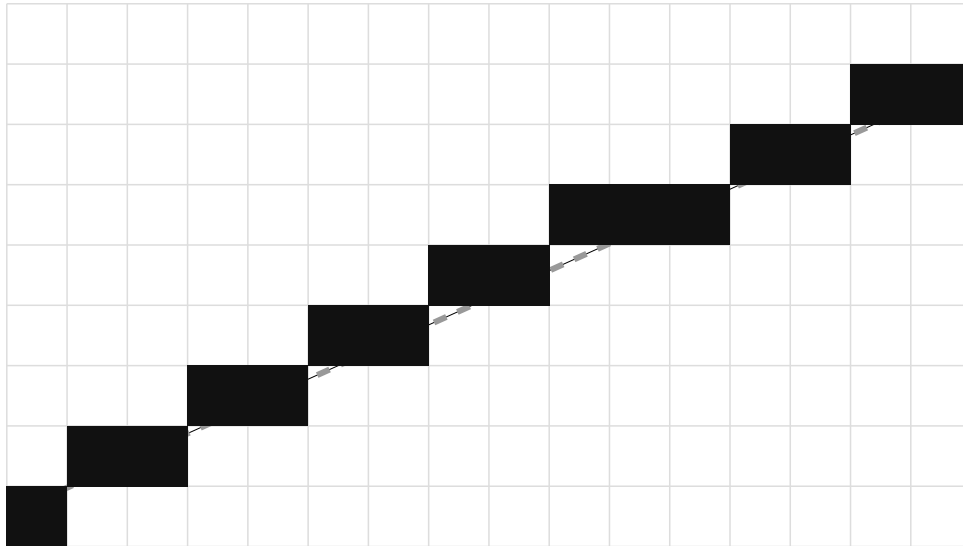
Both are important.

16. Aliasing: why lines look jagged

A mathematical line can be perfectly straight. But after scan conversion, it becomes a staircase of square pixels.

This visible stair-step effect is called **aliasing**.

Aliasing: smooth geometry becomes a staircase



A jagged edge is not a bug in one algorithm; it is a consequence of representing continuous shapes on a finite

Aliasing occurs because many different continuous lines may produce similar discrete pixel patterns. Fine continuous detail is being represented with limited samples.

Trivia: aliasing is not only in graphics

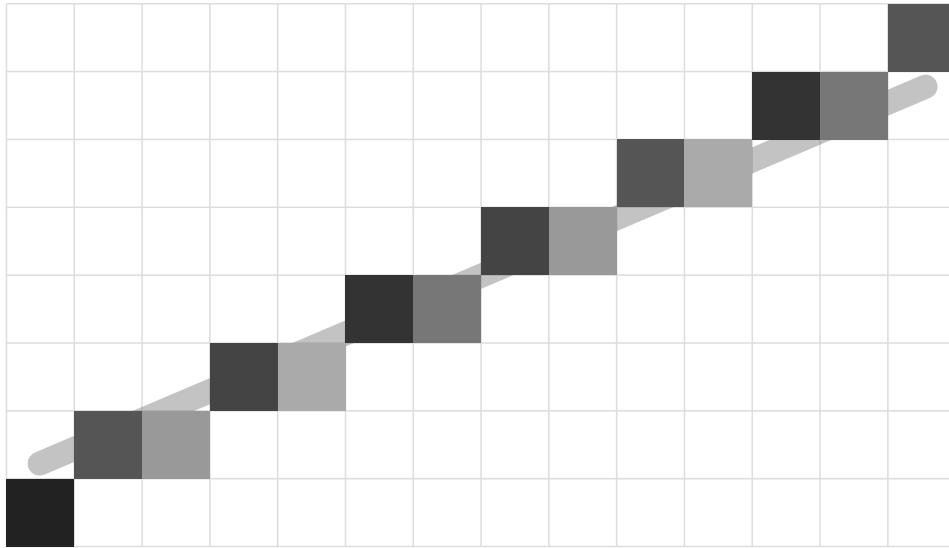
Aliasing also appears in audio and signal processing. If a sound wave is sampled too slowly, high-frequency information can appear as a false lower-frequency signal. In graphics, undersampling can create jagged edges, flickering, moiré patterns, and shimmering textures.

17. Anti-aliasing: making edges smoother

One way to reduce aliasing is to avoid the all-or-nothing decision of turning a pixel fully on or fully off.

Instead, we estimate how much of a pixel is covered by the ideal line and assign intensity accordingly.

Anti-aliasing: use partial intensity for partial coverage



Instead of only black or white pixels, edge pixels get intermediate shades.

For example:

```
high coverage    → dark pixel  
medium coverage → gray pixel  
low coverage     → light pixel  
no coverage      → background pixel
```

This makes the line look smoother to the human eye.

Application note

Modern text rendering depends heavily on anti-aliasing. Without anti-aliasing, letters on a screen would look blocky and harsh, especially at small font sizes.

18. Real-world applications

18.1 Fonts and text rendering

A font character may be stored as vector outlines. Before the user sees it, the outline must be rasterized into pixels. Anti-aliasing and hinting are used to make small text readable.

18.2 CAD and engineering drawing

CAD systems draw many precise lines, curves, arcs, and annotations. Scan conversion affects how cleanly these appear on screen and paper.

18.3 Digital maps

Roads, rivers, borders, and routes are often represented as geometric paths. They must be scan-converted to display them on phones and web maps.

18.4 Medical and scientific visualization

Contour lines, measurement markers, segmentation boundaries, and overlays are often drawn on top of image data.

18.5 Games and real-time graphics

Modern GPUs rasterize projected triangles into pixels. Line drawing is not the whole story, but it teaches the same transition from continuous geometry to discrete screen samples.

19. Common mistakes

Mistake 1: Thinking a pixel is a mathematical point

A pixel is an area. Algorithms often use its center as a simplified representative.

Mistake 2: Forgetting the slope case

The simple Bresenham formula shown earlier assumes $0 \leq m \leq 1$. Other slopes require adaptation.

Mistake 3: Confusing dx and dy

Always calculate:

$$\begin{aligned} dx &= x_2 - x_1 \\ dy &= y_2 - y_1 \end{aligned}$$

Then check the slope condition.

Mistake 4: Updating y every time

For $0 \leq m \leq 1$, x changes every step, but y changes only when the decision variable says so.

Mistake 5: Thinking anti-aliasing changes the geometry

Anti-aliasing does not change the mathematical line. It changes the pixel intensities used to represent that line.

21. Short questions

1. What is scan conversion?
 2. Why does a mathematical line need approximation on a raster display?
 3. What is the main idea of DDA line drawing?
 4. Why is Bresenham's algorithm faster than DDA on simple hardware?
 5. In Bresenham's algorithm for $0 \leq m \leq 1$, what are the two possible next pixels?
 6. What is the purpose of the decision variable?
 7. Why do jagged edges appear in raster graphics?
 8. What is anti-aliasing?
 9. Why is scan conversion important in modern 3D graphics?
 10. What changes when the line slope is greater than 1?
-

22. Worked exam-style problem

Question: Use Bresenham's line algorithm to find the pixels for a line from $(1, 1)$ to $(8, 5)$.

Solution:

$$\begin{aligned}dx &= 8 - 1 = 7 \\dy &= 5 - 1 = 4 \\p_0 &= 2dy - dx = 8 - 7 = 1\end{aligned}$$

Start with:

$(1, 1)$

Step	p before decision	Decision	Pixel
0	1	start	(1, 1)
1	1	NE	(2, 2)
2	$1 + 8 - 14 = -5$	E	(3, 2)
3	$-5 + 8 = 3$	NE	(4, 3)
4	$3 + 8 - 14 = -3$	E	(5, 3)
5	$-3 + 8 = 5$	NE	(6, 4)
6	$5 + 8 - 14 = -1$	E	(7, 4)
7	$-1 + 8 = 7$	NE	(8, 5)

So the pixels are:

$(1,1)$, $(2,2)$, $(3,2)$, $(4,3)$, $(5,3)$, $(6,4)$, $(7,4)$, $(8,5)$

23. Practice problems

1. Use DDA to draw a line from $(3, 4)$ to $(12, 8)$.
 2. Use Bresenham's algorithm to draw a line from $(2, 3)$ to $(9, 6)$.
 3. Use Bresenham's algorithm to draw a line from $(1, 2)$ to $(7, 10)$. First identify whether the slope is greater than 1.
 4. Explain why the same mathematical line may look different on displays of different resolutions.
 5. Draw a 10×10 grid and manually show the aliasing effect for a shallow line.
 6. Explain one real application where anti-aliasing is important.
-

24. Key takeaways

Lecture 3 summary

1. Scan conversion converts continuous geometry into discrete pixels.

2. DDA uses incremental floating-point steps and rounding.

3. Bresenham uses an integer decision variable for efficient line drawing.

4. Aliasing is inevitable on a pixel grid; anti-aliasing reduces visual jaggies.

- Scan conversion converts mathematical geometry into pixels.
- A raster display cannot show an ideal continuous line directly.
- DDA uses incremental real-valued steps and rounding.
- Bresenham's algorithm uses a decision variable and integer arithmetic.

- For $0 \leq m \leq 1$, each step moves in x , and the algorithm decides whether to also move in y .
 - Aliasing happens because continuous geometry is represented on a discrete pixel grid.
 - Anti-aliasing improves visual quality by using intermediate pixel intensities.
-

25. Suggested reading

1. Donald Hearn and M. Pauline Baker, *Computer Graphics with OpenGL*.
2. Steven Harrington, *Computer Graphics: A Programming Approach*.
3. Foley, van Dam, Feiner, and Hughes, *Computer Graphics: Principles and Practice*.
4. Any introductory OpenGL/WebGL rasterization tutorial for modern context.